MIST.cash

DOCUMENTATION · GUIDE · PAPER

polyring toolkit

Fast polynomial ring multiplications in gnark. Verify a Groth16 proof inside your circuit with up to 38% fewer constraints.

Contents

01 Overview 3

A recursive verifier is mostly pairing. 4

Check the whole step, not every multiplication. 4

A third less work for the prover. 4

One proof that carries another. 5

Cheaper to prove, dearer to verify on the EVM. 5

A few lines in Define. 6

Before you ship. 7

02 Guide 8

Install 9

Prove the inner statement 9

Bake in the inner key 9

Write the outer circuit 10

Compile and set up 10

Assign and prove 11

Export the Solidity verifier 11

Test it 12

Hand this to your agent 13

03 Paper 14

Abstract 15

One polynomial identity per Miller loop step 15

A third off the pairing, in gnark and on Starknet 16

Bigger numbers, same ratio 16

Not just pairings 17

Acknowledgements 17

CHAPTER 01

01

Overview

polyring is the Polynomial Ring Toolkit for gnark. Its first use is recursion: verify a Groth16 proof inside your circuit with up to 38% fewer constraints than gnark's own pairing.

IN THIS CHAPTER

— A recursive verifier is mostly pairing.

— Check the whole step, not every multiplication.

— A third less work for the prover.

— One proof that carries another.

— Cheaper to prove, dearer to verify on the EVM.

— A few lines in Define.

— Before you ship.

THE PROBLEM

A recursive verifier is mostly pairing.

To check a Groth16 proof inside a circuit, you compute a pairing in emulated field arithmetic. With gnark's pairing, that is 83% of the whole verifier.

gnark's verifier, BN254 inner proof 791,400 constraints



gnark's Miller loop and final exponentiation Inputs, point checks

THE IDEA

Check the whole step, not every multiplication.

A pairing loops over dozens of steps, each a handful of multiplications in a degree-12 field. gnark proves every multiplication. polyring writes each step as one polynomial identity and checks all of them together at a single random point.

How it works, in the paper →

$$F^2 \cdot L_1 L_2 L_3 = Q \cdot P_{12} + R$$

132
constraints per loop step, field multiplication by multiplication

41
constraints per loop step as one identity

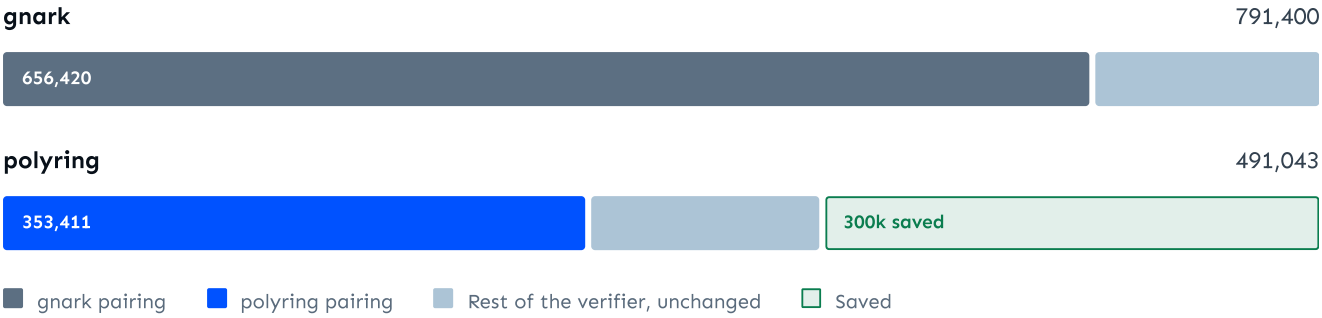
THE RESULT

A third less work for the prover.

Same circuit, same statement. Only the pairing check changes, and the rest of the verifier stays as it was.

−38%
constraints
791,400 → 491,043

−34%
witness solve time
340 → 223 ms, M3 Pro



WHAT IT'S FOR

One proof that carries another.

Wrap and compress

Fold a proof, plus your own checks on it, into one BN254 Groth16 proof.

BLS12-381 on Ethereum

Verify a BLS12-381 proof inside a BN254 circuit. The EVM's pairing precompile checks the result.

Any pairing check

The ring pairing isn't tied to Groth16. A general two-pair check costs 31% fewer constraints on BN254.

THE TRADE

Cheaper to prove, dearer to verify on the EVM.

polyring's proof carries three commitments instead of one, so the Solidity verifier does more work: about 460k gas per `verifyProof` call, against 359k with gnark's pairing. Pick it when proving is your main cost: many proofs, modest hardware, or a proving service that bills by the constraint.

Where a VM runs the pairing itself, verifying gets cheaper too. Garaga on Starknet uses the paper's technique and needs **37%** fewer VM steps.

PROVER

Less to prove

Constraints -38%
791,400 → 491,043

Committed wires -32%
589,535 → 398,959

EVM VERIFIER

More to check

Verification gas +28%
≈359k → ≈460k per `verifyProof` call

Proof size +33%
384 → 512 bytes

IN YOUR CIRCUIT

A few lines in `Define`.

Bake the inner verifying key into your circuit, verify the inner proof first, then add your own constraints. polyring also generates the Solidity verifier for the outer proof.

outer.go

```
type Outer struct {
    InnerProof  recursion.Proof[Fp, G2]     `gnark:",secret"`
    InnerPublic recursion.PublicWitness[Fr] `gnark:",public"`
    vk          *recursion.VerifyingKey
}

func (c *Outer) Define(api frontend.API) error {
    v, err := recursion.NewVerifier[Fp, Fr, G2, GTEl](api, c.vk)
    if err != nil {
        return err
    }
    if err := v.AssertProof(c.InnerProof, c.InnerPublic); err != nil {
        return err
    }
    // your own constraints
    return nil
}
```

SCOPE OF V0.1

Before you ship.

Unaudited	A bug in an in-circuit pairing breaks soundness. Use it for research and on testnets.
One fixed inner key	The inner verifying key is baked in when the circuit compiles.
One proof at a time	Each outer proof verifies one inner proof. Batching several is planned.
No nested recursion	An outer polyring proof can't be an inner proof yet, and inner circuits can't use `api.Commit`.

CHAPTER 02

02

Guide

Eight steps from an inner proof to a contract that verifies the recursive proof. All code comes from a module that runs the whole flow in about a minute.

IN THIS CHAPTER

— Install
— Prove the inner statement
— Bake in the inner key
— Write the outer circuit
— Compile and set up
— Assign and prove
— Export the Solidity verifier
— Test it
— Hand this to your agent

1 Install

polyring needs a gnark fix that isn't upstream yet. Go ignores `replace` directives in dependencies, so copy polyring's into your `go.mod`.

shell

```
go get github.com/mistcash/polyring
```

go.mod

```
// Copy the current line from polyring's go.mod; it moves with polyring.
replace github.com/consensys/gnark v0.16.4 => github.com/mistcash/tiny-gnark v0.0.0-20261018205550-0a3abd2ee477
```

Then run `go mod tidy`. Check polyring's go.mod for the current line.

2 Prove the inner statement

Any gnark circuit over BN254 or BLS12-381, proved with stock Groth16. Keep the verifying key, the proof and the public witness.

inner.go

```
// Error handling omitted. For a BLS12-381 inner proof use ecc.BLS12_381.
ccs, err := frontend.Compile(ecc.BN254.ScalarField(), r1cs.NewBuilder, &InnerCircuit{})
pk, innerVK, err := groth16.Setup(ccs)
full, err := frontend.NewWitness(innerAssignment, ecc.BN254.ScalarField())
innerProof, err := groth16.Prove(ccs, pk, full)
innerPublic, err := full.Public()
```

To try the flow first, `mimc.Prove(ecc.BN254)` from the examples does all of this for a MiMC circuit.

> **No commitments in the inner circuit.** It must not call `api.Commit`, directly or through a gadget that does (range checks, lookup tables and emulated arithmetic can). Step 3 rejects such keys.

3 Bake in the inner key

The inner verifying key becomes circuit constants. `NewVerifyingKey` checks its points, precomputes what it can, and rejects keys with commitments.

outer.go

```
vk, err := recursion.NewVerifyingKey(innerVK)
```

4 Write the outer circuit

Add the inner proof and public inputs as fields, keep the key in an unexported field, and verify the proof first in `Define`. Your own constraints follow.

outer.go

```
// The inner proof's curve. For BLS12-381 use emparams.BLS12381Fp /
// BLS12381Fr and sw_bls12381.G2Affine / GTEl.
type (
    Fp   = emparams.BN254Fp // inner curve base field
    Fr   = emparams.BN254Fr // inner curve scalar field
    G2   = sw_bn254.G2Affine
    GTEl = sw_bn254.GTEl
)

type Outer struct {
    InnerProof  recursion.Proof[Fp, G2]     `gnark:",secret"`
    InnerPublic recursion.PublicWitness[Fr] `gnark:",public"`
    Nonce       frontend.Variable           `gnark:",public"` // your own inputs

    vk *recursion.VerifyingKey // the inner key, baked in as constants
}

// NewOuter sets the key and sizes the inner public inputs.
// Use it for both compiling and assigning.
func NewOuter(vk *recursion.VerifyingKey) *Outer {
    return &Outer{InnerPublic: recursion.NewBN254Circuit(vk).PublicWitness, vk: vk}
}

func (c *Outer) Define(api frontend.API) error {
    // First line: the ring checker must exist before any emulated field.
    v, err := recursion.NewVerifier[Fp, Fr, G2, GTEl](api, c.vk)
    if err != nil {
        return err
    }
    if err := v.AssertProof(c.InnerProof, c.InnerPublic); err != nil {
        return err
    }
    api.AssertIsDifferent(c.Nonce, 0) // your own constraints
    return nil
}
```

> **Why first?** The ring checks register range checks of their own, and gnark runs deferred checks in registration order.

5 Compile and set up

The outer circuit is always over BN254, whatever the inner curve. Compile with the same `vk` you assign with.

main.go

```
ccs, err := frontend.Compile(ecc.BN254.ScalarField(), r1cs.NewBuilder, NewOuter(vk))
pk, outerVK, err := groth16.Setup(ccs) // about 491k constraints
```

`groth16.Setup` is a local trusted setup, fine for tests. Keys that secure anything need a ceremony.

6 Assign and prove

The circuit helper converts the native proof and public witness into circuit values. Copy them into your assignment.

main.go

```
tmp := recursion.NewBN254Circuit(vk)
if err := tmp.Assign(innerProof, innerPublic); err != nil {
    return err
}
assignment := NewOuter(vk)
assignment.InnerProof, assignment.InnerPublic = tmp.Proof, tmp.PublicWitness
assignment.Nonce = 7 // your own inputs

full, err := frontend.NewWitness(assignment, ecc.BN254.ScalarField())
public, err := full.Public()
proof, err := groth16.Prove(ccs, pk, full,
    gnarksolidity.WithProverTargetSolidityVerifier(backend.GROTH16))
err = groth16.Verify(proof, outerVK, public,
    gnarksolidity.WithVerifierTargetSolidityVerifier(backend.GROTH16))
```

The Solidity target derives commitment challenges with Keccak, as the contract does. Leave it out if you only verify off chain.

7 Export the Solidity verifier

Use polyring's exporter. gnark's own supports one commitment, and polyring proofs carry three.

main.go

```
bnVK := outerVK.(*groth16bn254.VerifyingKey)
bnProof := proof.(*groth16bn254.Proof)

f, err := os.Create("Verifier.sol")
err = solidity.ExportSolidity(bnVK, f)          // polyring's lib/solidity
proofBytes := solidity.MarshalSolidity(bnProof) // 512 bytes

// Public inputs: the outer public witness, minus the values the
// contract derives from the commitments itself.
vec := public.Vector().(fr.Vector)
n := len(bnVK.G1.K) - 1 - len(bnVK.PublicAndCommitmentCommitted)
inputs := make([]*big.Int, n)
for i := range inputs {
    inputs[i] = vec[i].BigInt(new(big.Int))
}
// On chain: Verifier.verifyProof(proofBytes, inputs) reverts unless valid.
```

Each inner public input enters as four 64-bit limbs. With one inner input and a nonce, this example passes five.

8 Test it

Check the assignment before paying for setup, then run polyring's end-to-end tests as a reference.

outer_test.go

```
// Seconds, no setup: does the assignment satisfy the circuit?
err := test.IsSolved(NewOuter(vk), assignment, ecc.BN254.ScalarField())
```

shell, in a polyring checkout

```
go test ./examples/recursion/ -run TestGroth16Verifier -v   # both inner curves, tampered proofs rejected
go test ./examples/recursion/ -run TestOuterOnChain -v      # prove, deploy, verify on a simulated EVM (needs solc)
```

▶ **Imports used in this guide**

FOR CODING AGENTS

Hand this to your agent

Paste the prompt below, or point your agent at `llms.txt`, which adds the full tested code. These are the rules agents get wrong without help.

prompt

```
You are adding recursive Groth16 verification to a Go project with polyring
(github.com/mistcash/polyring), a gnark library. Read
https://polyring.mist.cash/llms.txt first, then follow these rules.

1. go.mod: require github.com/mistcash/polyring and copy polyring's gnark
   replace directive verbatim from its go.mod. Go ignores replace directives
   in dependencies, and polyring's circuits need that gnark fork.
2. The inner circuit (BN254 or BLS12-381, Groth16) must draw no BSB22
   commitments: no api.Commit, directly or through a gadget.
   recursion.NewVerifyingKey(innerVK) returns an error if it does.
3. In the outer circuit struct use recursion.Proof[Fp, G2] (secret) and
   recursion.PublicWitness[Fr] (public), and keep the
   *recursion.VerifyingKey in an unexported field.
   BN254 inner:     emparams.BN254Fp, emparams.BN254Fr,
                    sw_bn254.G2Affine, sw_bn254.GTEl
   BLS12-381 inner: emparams.BLS12381Fp, emparams.BLS12381Fr,
                    sw_bls12381.G2Affine, sw_bls12381.GTEl
4. recursion.NewVerifier[Fp, Fr, G2, GTEl](api, vk) must be the first
   statement in Define, before anything creates an emulated field. Then
   call AssertProof, then add the project's own constraints.
5. Build the compile placeholder and the assignment from the same vk. Size
   the public inputs with recursion.NewBN254Circuit(vk).PublicWitness (or
   NewBLS12381Circuit) and fill the assignment through that helper's Assign.
6. Compile over ecc.BN254.ScalarField() with r1cs.NewBuilder. When the proof
   goes on chain, prove with gnark's
   solidity.WithProverTargetSolidityVerifier(backend.GROTH16).
7. Export the contract with polyring's lib/solidity.ExportSolidity and pack
   the proof with lib/solidity.MarshalSolidity. Never gnark's own exporter:
   it supports one commitment and polyring proofs carry three.
8. Check the assignment with test.IsSolved before running setup.

Do not claim the EVM verifier is cheaper than gnark's: it costs about 460k
gas against 359k; the saving is 38% fewer prover constraints. Do not
attempt nested recursion, inner proofs with commitments, or a verifying key
that varies at runtime: v0.1 does not support them. polyring is unaudited.
```

CHAPTER 03

03

Paper

Pairing Proofs and Polynomial Ring Toolkit, to appear in IACR TCHES 2027. Shramee Srivastav, MIST.cash.

IN THIS CHAPTER

- Abstract
- One polynomial identity per Miller loop step
- A third off the pairing, in gnark and on Starknet
- Bigger numbers, same ratio
- Not just pairings
- Acknowledgements

ABSTRACT

Elliptic curve pairings are fundamental to modern zero-knowledge proof systems, including Groth16, PLONK, and KZG polynomial commitments. In resource-constrained execution environments (Ethereum VM, BitVM, blockchain smart contracts), where computation is metered by gas fees, and in arithmetized computation systems (R1CS, Plonkish, Cairo AIR), where each computational step imposes significant prover overhead, the cost of pairing computation remains a practical bottleneck. We present a public-coin interactive proof system that consolidates the Miller loop iterations and final exponentiation into a single polynomial relation. The relation can then be verified as polynomial identities, greatly reducing the pairing costs. We present a full implementation in gnark (the industry-standard framework for arithmetized circuit development) as a reusable polynomial ring toolkit in the emulated package. Benchmarked on a 3-pairing product check (the pairing workload of a Groth16 verification), our gnark implementation achieves over 32% reduction in constraints, over 50% reduction in committed elements, and reductions in both compile-time and solver memory for both BN254 and BLS12-381 curves. This enables practical pairing verification in recursive proof systems and resource-constrained environments.

METHOD

One polynomial identity per Miller loop step

$$F(x)^2 \cdot L_1(x)\, L_2(x)\, L_3(x) = Q(x) \cdot P_{12}(x) + R(x)$$

Earlier work checks every $\mathbb{F}_p^{12}$ multiplication as its own identity and commits each remainder. This work checks a whole loop step at once, then batches every step and the final exponentiation into one check at a random point. Soundness error stays below 2^{-244} for BN254.

F is the loop accumulator, L_1 to L_3 the line functions of three pairs, and $P_{12} = x^{12} - 18x^6 + 82$ the BN254 extension modulus. Builds on El Housni, Feltroidprime, and Novakovic and Eagen's On Proving Pairings.

Constraints per loop step, three pairs



Paper, Table 1, zero bit.

RESULTS

A third off the pairing, in gnark and on Starknet

Table 7: 3-pairing product check in gnark, baseline against this work. Apple M3 Pro.

METRIC	BN254	BLS12-381
R1CS		
Constraints	-35.7%	-32.5%
Committed $\mathbb{F}_q$ elements	-52.8%	-54.5%
Compile time	-40.5%	-38.3%
Solve time	-39.4%	-33.7%
PLONK (SCS)		
Constraints	-35.0%	-33.2%
Compile time	-43.6%	-36.9%
Solve time	-41.9%	-38.0%

Table 6: Garaga on Starknet, 3-pairing check with final exponentiation, in production.

METRIC	BN254	BLS12-381
CairoVM steps	-37.1%	-32.7%
Committed elements	-50.7%	-41.6%
MULMOD	-33.9%	-31.5%
ADDMOD	-30.2%	-25.4%

Fewer committed elements means less to hash and less hint calldata per verification.

▶ **Table 7 in full**

PAPER TO LIBRARY

Bigger numbers, same ratio

The paper measured a bare 3-pairing check on a gnark fork. Two gnark advisories published since then, GHSA-3mvx-pp85-pm65 and GHSA-fwf3-jmp7-gmrj, added range checks to every emulated multiplication. Both columns grew, and the saving held.

The paper's fork stays frozen to reproduce its tables. polyring builds on the fixed gnark.

R1CS constraints, BN254



* Preliminary: measured, benchmark not yet in the tree.

THE TOOLKIT

Not just pairings

The protocol ships as a toolkit for any emulated field and any monic modulus. Register a ring, submit products, and the checker proves them all with two commitments when `Define` returns. The paper names lattice schemes such as ML-KEM and ML-DSA as a next application.

std/polyring

```
prc := polyring.NewPolyRingChecker[emulated.BN254Fp](api) // before any other emulated field
modPoly := prc.MakePoly(82, 0, 0, 0, 0, 0, -18, 0, 0, 0, 0, 0, 1)
ring := prc.NewPolyRingCheck(modPoly, nil)
res, _ := prc.MulPolyRings(ring, A, B, C) // (A·B·C) mod modPoly, checked when Define returns
acc := prc.NewPolyRingAccumulator(ring, targetDeg)
acc.Mul(A).Mul(B).Sqr() // enqueue (A·B)², nothing multiplied yet
rem := acc.Eval()        // one deferred check for the whole chain
```

ACKNOWLEDGEMENTS

Youssef El Housni and Ivo Kubjas (Linea, Consensys) for discussions and encouragement, Feltroidprime for discussions on his technique, and StarkWare and the Starknet Foundation for supporting an earlier version of the technique.

COLOPHON

polyring toolkit

This book collects the polyring website: the overview, the step-by-step guide, and the paper summary. The library is Apache-2.0 licensed. Version 0.1 is unaudited; use it for research and on testnets.

Source	github.com/mistcash/polyring
Website	polyring.mist.cash
Paper	IACR TCHES 2027 · polyring.mist.cash/paper.pdf
Author	Shramee Srivastav, MIST.cash
Type	Josefin Sans and Sen, set in the MIST design system